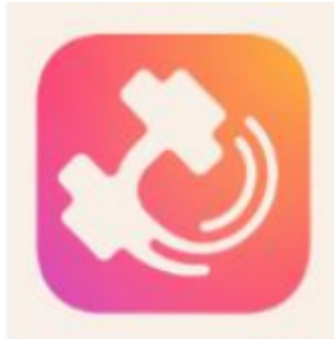




GYMTRACKER

INFORME DE CIERRE ASIGNATURA FERRAMENTAS DE DESARROLLO

Fecha: 22/12/2025

Miembros del grupo:

Enoc Arias Buján

Manuel Cabaleiro Fernández

Jacobo García Pérez

Daniel García Varela

Pablo Masián Carro

1. Forma de trabajo

El desarrollo de la aplicación *Gymtracker* se realizó de forma colaborativa apoyándose en un conjunto de herramientas de desarrollo que dieron soporte a todo el ciclo de vida del software. Estas herramientas permitieron organizar el trabajo del equipo, facilitar la colaboración, automatizar tareas repetitivas y controlar la calidad del producto a lo largo de las distintas iteraciones del proyecto.

1.1. Entorno de desarrollo

Como entorno de desarrollo integrado se utilizó **Visual Studio Code (VS Code)**, que fue la herramienta principal durante toda la práctica. Este IDE permitió trabajar de manera unificada tanto en el desarrollo del frontend como del backend, integrando funcionalidades como el control de versiones, la ejecución de comandos, la gestión de dependencias y la ejecución de pruebas. El uso de un entorno común para todo el equipo favoreció la productividad y redujo problemas derivados de diferencias de configuración.

1.2. Control de versiones

El control de versiones del proyecto se realizó mediante **Git**, utilizando un repositorio remoto alojado en **GitLab**. Se adoptó una estrategia de ramas basada en una rama *main*, destinada a las versiones estables y entregas oficiales de cada iteración y una rama *develop*, utilizada como rama de integración del trabajo del equipo, donde día a día se integraban las ramas locales de trabajo de los distintos issues.

El desarrollo de nuevas funcionalidades y correcciones se llevó a cabo en ramas individuales, que posteriormente se fusionaban en *develop* y, tras su validación completa, en *main*. Este enfoque permitió mantener la estabilidad del proyecto, facilitar el trabajo concurrente y disponer de un historial claro de los cambios realizados.

1.3. Gestión de tareas e incidencias

Para la planificación y el seguimiento del trabajo se utilizó **Redmine** como herramienta de issue tracking. En Redmine se registraron las tareas, incidencias y mejoras del proyecto, permitiendo asignar responsables, establecer prioridades, estimar tiempos de trabajo y asociar las incidencias a versiones concretas del sistema.

El uso de esta herramienta facilitó la organización del trabajo del equipo, el seguimiento del progreso del proyecto y la coordinación entre los distintos integrantes durante las iteraciones de desarrollo.

1.4. Automatización del build

El backend de la aplicación se desarrolló como un proyecto **Maven**, lo que permitió automatizar el proceso de construcción del software. A través del fichero de configuración correspondiente, se definieron las dependencias del proyecto, el ciclo de vida del build y la ejecución automática de pruebas.

Esta automatización garantizó que el proyecto pudiera compilarse y verificarse de forma consistente en distintos entornos.

1.5. Integración continua

Se empleó **Jenkins** como herramienta de **integración continua**, encargada de automatizar la ejecución del proceso de build y validación del código al integrar cambios en el repositorio. De este modo, fue posible detectar de forma temprana errores de compilación o fallos en las pruebas, mejorando la estabilidad del proyecto a lo largo de su desarrollo.

1.6. Inspección continua y calidad del código

Para el control de la calidad del código se utilizó **SonarQube** como herramienta de inspección continua. Esta herramienta permitió analizar el código fuente del proyecto, detectando posibles errores, code smells y problemas de mantenibilidad, así como evaluar la cobertura de las pruebas automatizadas.

La información proporcionada por SonarQube sirvió como apoyo para mejorar progresivamente la calidad del código, corrigiendo algunos de los problemas detectados durante el desarrollo.

1.7 Pruebas de rendimiento

Durante el desarrollo del proyecto se llevaron a cabo **pruebas de rendimiento** con el objetivo de evaluar el comportamiento del backend bajo escenarios de carga concurrente. Para ello se diseñaron tests que simulaban el acceso simultáneo de múltiples usuarios a los principales endpoints de la aplicación, reproduciendo operaciones habituales como la consulta de rutinas, el inicio de sesión y la carga del feed.

Estas pruebas permitieron analizar los tiempos de respuesta y la capacidad del sistema para soportar concurrencia, aportando información relevante sobre los límites de rendimiento de la aplicación y sobre el impacto del uso de una base de datos embebida en entornos de alta carga.

1.8. Pruebas funcionales automatizadas

Durante el desarrollo de la aplicación se implementaron **pruebas funcionales automatizadas** con el objetivo de verificar el correcto funcionamiento de los distintos componentes del sistema y detectar errores de forma temprana durante el desarrollo.

En el backend, desarrollado en Java con Spring, se empleó **JUnit** como framework principal para la definición y ejecución de pruebas unitarias e de integración. Para aislar el comportamiento de los componentes bajo prueba y simular dependencias externas, se utilizó **Mockito**, permitiendo crear objetos simulados (mocks) y centrar los tests en la lógica específica de cada componente.

Adicionalmente, se empleó **JaCoCo** como herramienta de medición de cobertura de código en el backend, integrada en el ciclo de construcción del proyecto mediante Maven. Esta herramienta permitió evaluar qué partes del código eran ejecutadas durante las pruebas, aportando información útil para mejorar la calidad y fiabilidad del sistema.

El uso de pruebas automatizadas formó parte del flujo habitual de desarrollo, contribuyendo a mejorar la estabilidad del proyecto y a detectar errores antes de la integración de cambios en las ramas principales del repositorio.

1.9. Containerización y despliegue automatizado

Durante el desarrollo del proyecto se trabajó en la **preparación de la aplicación para su despliegue mediante contenedores**, utilizando **Docker** para encapsular el entorno de ejecución y facilitar la portabilidad del sistema entre distintos entornos.

Asimismo, se abordó la **integración del proceso de despliegue dentro del sistema de integración continua**, empleando **Jenkins** como herramienta de automatización. Esta configuración permitió sentar las bases para un flujo de **CI/CD**, orientado a automatizar tareas de construcción, validación y despliegue de la aplicación.

1.10. Visión global del flujo de trabajo

El uso combinado de todas las herramientas empleadas en el proyecto permitió establecer un flujo de trabajo estructurado y coherente, alineado con prácticas habituales en entornos profesionales de desarrollo de software. A lo largo del desarrollo se integraron herramientas de control de versiones, gestión de tareas, automatización del build, integración continua, pruebas automatizadas, análisis de calidad, pruebas de rendimiento y preparación del despliegue.

Este enfoque facilitó el trabajo colaborativo, el control y la trazabilidad de los cambios, la detección temprana de errores y la mejora progresiva de la calidad del sistema.

2. Entregas

2.1. Organización de las entregas

El desarrollo de la aplicación se realizó de forma iterativa, planificando el trabajo en distintas fases que culminaban en 4 entregas parciales del producto. Para la gestión de estas entregas se utilizó un repositorio GitLab, que actuó como punto central de integración y almacenamiento del código fuente.

La rama `main` se empleó como rama estable del proyecto, reservada para las versiones entregables y defensas, mientras que la rama `develop` se utilizó como rama de integración del trabajo del equipo. Antes de cada entrega, los cambios se consolidaban en la rama principal, garantizando que la versión presentada correspondiera a un estado estable y funcional del sistema.

2.2. Entregas iterativas y evolución del producto

Cada entrega correspondió a una iteración del proyecto, en la que se incorporaron nuevas funcionalidades, mejoras sobre desarrollos previos y correcciones de errores detectados en iteraciones anteriores. Este enfoque permitió una evolución progresiva del producto, facilitando la adaptación a los requisitos y la mejora continua de la aplicación.

Previo a la consolidación de cada entrega, el código fue validado mediante pruebas automatizadas y procesos de integración continua, reduciendo el riesgo de introducir errores en las versiones entregables. De este modo, cada iteración supuso un incremento en la estabilidad y madurez del sistema.

3. Requisitos

3.1. Origen de los requisitos

Los requisitos de la aplicación Gymtracker fueron definidos en el marco de la asignatura de Metodologías de Desarrollo, a partir del enunciado de la práctica y de la planificación realizada durante el curso. En este contexto, la profesora de la asignatura actuó como cliente del proyecto, siendo la encargada de definir los requisitos funcionales de la aplicación, así como de proponer correcciones y cambios a lo largo de las distintas iteraciones.

Desde la asignatura de Ferramentas de Desenvolvimento, estos requisitos se tomaron como punto de partida, centrándose el trabajo en su implementación y soporte técnico mediante el uso de herramientas de desarrollo.

3.2. Gestión de los requisitos durante el desarrollo

La gestión de los requisitos se realizó de forma indirecta mediante Redmine, donde los requisitos definidos se descompusieron en tareas técnicas e incidencias asociadas al desarrollo. Estas tareas se asignaron a responsables y se vincularon a versiones del proyecto, permitiendo el seguimiento del progreso y la adaptación del desarrollo a los cambios solicitados por el cliente a lo largo de las iteraciones.

3.3. Validación de los requisitos

La validación de los requisitos se llevó a cabo mediante pruebas funcionales automatizadas y procesos de integración continua. Se utilizaron pruebas en el backend y frontend para verificar el correcto funcionamiento de las funcionalidades implementadas, apoyadas por Jenkins para la ejecución automática de los procesos de validación. Adicionalmente, el análisis de calidad del código y las pruebas de rendimiento contribuyeron a asegurar la estabilidad y fiabilidad del sistema.

4. Incidencias

4.1. Qué incidencias surgieron

En este proyecto surgieron diversas incidencias a lo largo de las distintas iteraciones, tanto de carácter técnico como organizativo, que influyeron en el desarrollo de la aplicación Gymtracker.

En la primera iteración, la principal dificultad fue la conexión con la base de datos H2. Al tratarse de una tecnología con la que el equipo no estaba familiarizado, fue necesario

invertir tiempo en investigar su configuración e integración con Spring. Tras varias pruebas y ajustes, se consiguió establecer correctamente la conexión y disponer de un sistema funcional para el almacenamiento de los datos de la aplicación.

Durante las primeras semanas de trabajo con la metodología SCRUM, otra de las incidencias fue la falta de constancia en el cumplimiento de los objetivos semanales. Al inicio, al equipo le costó adaptarse al ritmo de trabajo iterativo y a la necesidad de completar las tareas planificadas en cada sprint. Con el avance del proyecto, se mejoró la organización interna y la distribución del trabajo, lo que permitió cumplir de forma más regular con los compromisos establecidos.

En relación con los requisitos funcionales, la visualización de los avatares de usuario supuso un reto importante. Aunque se consiguió gestionar correctamente el almacenamiento y recuperación de las imágenes, aparecieron problemas para mostrarlas de forma consistente en todas las vistas de la aplicación, como en la barra lateral y en el perfil del usuario. Finalmente, tras varios ajustes en el frontend, se logró una visualización correcta y homogénea.

En un sprint posterior, la introducción del sistema de notificaciones representó otra incidencia relevante. La gestión de eventos, su persistencia y la actualización en tiempo real en la interfaz requieren más esfuerzo del previsto inicialmente. Fue necesario replantear parte de la lógica y realizar pruebas adicionales para asegurar que las notificaciones funcionasen de forma fiable.

En la tercera iteración, la mayor dificultad fue el desarrollo del feed principal de la aplicación. La incorporación de un componente social con likes, comentarios y la visualización de entrenamientos de los usuarios seguidos implicó un cambio importante en la arquitectura y en el volumen de trabajo. Integrar todas estas funcionalidades de forma coherente y eficiente supuso un desafío que requirió más tiempo y coordinación entre backend y frontend.

Finalmente, en la cuarta iteración, la principal incidencia estuvo relacionada con la funcionalidad de “wrapped”. La necesidad de recopilar y procesar datos anuales, cuando hasta ese momento se trabajaba únicamente con datos mensuales, obligó a modificar consultas y lógica de negocio ya existente. Además, la representación de esta información en el frontend, en forma de diapositivas y con un diseño visual atractivo, resultó compleja para el equipo, al no estar acostumbrado a este tipo de implementaciones.

En conjunto, estas incidencias permitieron al equipo enfrentarse a problemas reales de desarrollo, aprender a resolverlos de forma colaborativa y mejorar tanto en aspectos técnicos como organizativos a lo largo del proyecto.

5. Estimaciones de planificación inicial vs. resultado final

5.1. Planificación inicial

Al inicio del proyecto, el equipo estimó que Gymtracker sería una aplicación de complejidad moderada, centrada principalmente en funcionalidades básicas de gestión de entrenamientos y usuarios, sin un fuerte componente social. Se preveía que el trabajo pudiera repartirse de forma relativamente flexible y que las tareas no completadas en una semana pudieran trasladarse a la siguiente sin un impacto significativo.

Asimismo, se asumió que la planificación semanal no sería tan estricta y que el volumen de trabajo por iteración sería manejable dentro del tiempo disponible.

5.2. Desarrollo real del proyecto

A medida que avanzó el proyecto, se comprobó que la aplicación tenía un grado de complejidad mayor del esperado. La incorporación de funcionalidades como el feed social, los likes, los comentarios, las notificaciones o el wrapped anual incrementó notablemente el esfuerzo necesario tanto en backend como en frontend.

La metodología SCRUM obligó a mantener un ritmo de trabajo constante, en el que las tareas no completadas en una semana se acumulaban con las nuevas del siguiente sprint. Esto evidenció la importancia de cumplir los objetivos semanales, ya que cualquier retraso tenía un efecto directo en la carga de trabajo posterior.

En la práctica, varias fases requirieron más tiempo del inicialmente estimado, especialmente aquellas relacionadas con:

- Integración de nuevas funcionalidades complejas.
- Resolución de incidencias técnicas.
- Ajustes derivados de cambios en los requisitos.

5.3. Comparación y desviaciones

De forma general, puede afirmarse que:

- El volumen de trabajo real fue superior al previsto inicialmente.
- Algunas tareas planificadas como simples resultaron ser más complejas.
- Hubo desviaciones en los tiempos de desarrollo de ciertas iteraciones, compensadas gracias al esfuerzo adicional del equipo.

Estas desviaciones se debieron principalmente a:

- Falta de experiencia previa con algunas tecnologías.

- Subestimación del impacto del componente social de la aplicación.
- Necesidad de rehacer o refinar partes del sistema ya implementadas.

5.4. Conclusión

Aunque la planificación inicial no se ajustó completamente a la realidad del proyecto, el equipo supo adaptarse a las circunstancias, reorganizar el trabajo y priorizar funcionalidades para poder cumplir con las entregas. Esta experiencia permitió comprender la importancia de realizar estimaciones realistas y de revisar periódicamente la planificación en función de la evolución real del proyecto.

6. Expectativas de calidad

6.1. Expectativas iniciales

Desde el inicio del proyecto, se estableció como objetivo desarrollar una aplicación:

- Funcional y estable.
- Fácil de usar para el usuario final.
- Con una interfaz clara y atractiva.
- Con un código mantenible y de calidad.
- Respalda por pruebas automatizadas y herramientas de análisis.

Se esperaba que el sistema cumpliera los requisitos definidos en la asignatura y que pudiera evolucionar de forma ordenada a lo largo de las iteraciones.

6.2. Criterios de calidad

Para evaluar la calidad del producto se tuvieron en cuenta los siguientes aspectos:

- Correcto funcionamiento de las funcionalidades principales.
- Ausencia de errores críticos.
- Usabilidad de la interfaz.
- Rendimiento aceptable bajo carga.

- Calidad del código medida mediante SonarQube.
- Cobertura de pruebas con JUnit y JaCoCo.
- Estabilidad del build mediante Jenkins.

6.3. Resultado obtenido

Al finalizar el proyecto, la aplicación Gymtracker cumple en gran medida con las expectativas iniciales:

- Las funcionalidades principales están implementadas y operativas.
- El sistema es estable y no presenta fallos críticos conocidos.
- La interfaz es usable y coherente con los objetivos de la aplicación.
- El código ha sido analizado y mejorado mediante herramientas de calidad.
- Las pruebas automatizadas han contribuido a detectar errores de forma temprana.
- Las pruebas de rendimiento han permitido evaluar los límites del sistema.

6.4. Valoración final y posibles mejoras

La valoración global de la calidad del proyecto es positiva, ya que se ha conseguido un producto funcional, estable y alineado con los requisitos establecidos.

No obstante, se identifican posibles mejoras futuras, como:

- Optimizar el rendimiento en escenarios de alta concurrencia.
- Aumentar la cobertura de pruebas automatizadas.
- Refinar ciertos aspectos visuales del frontend.
- Mejorar la gestión de errores y validaciones.

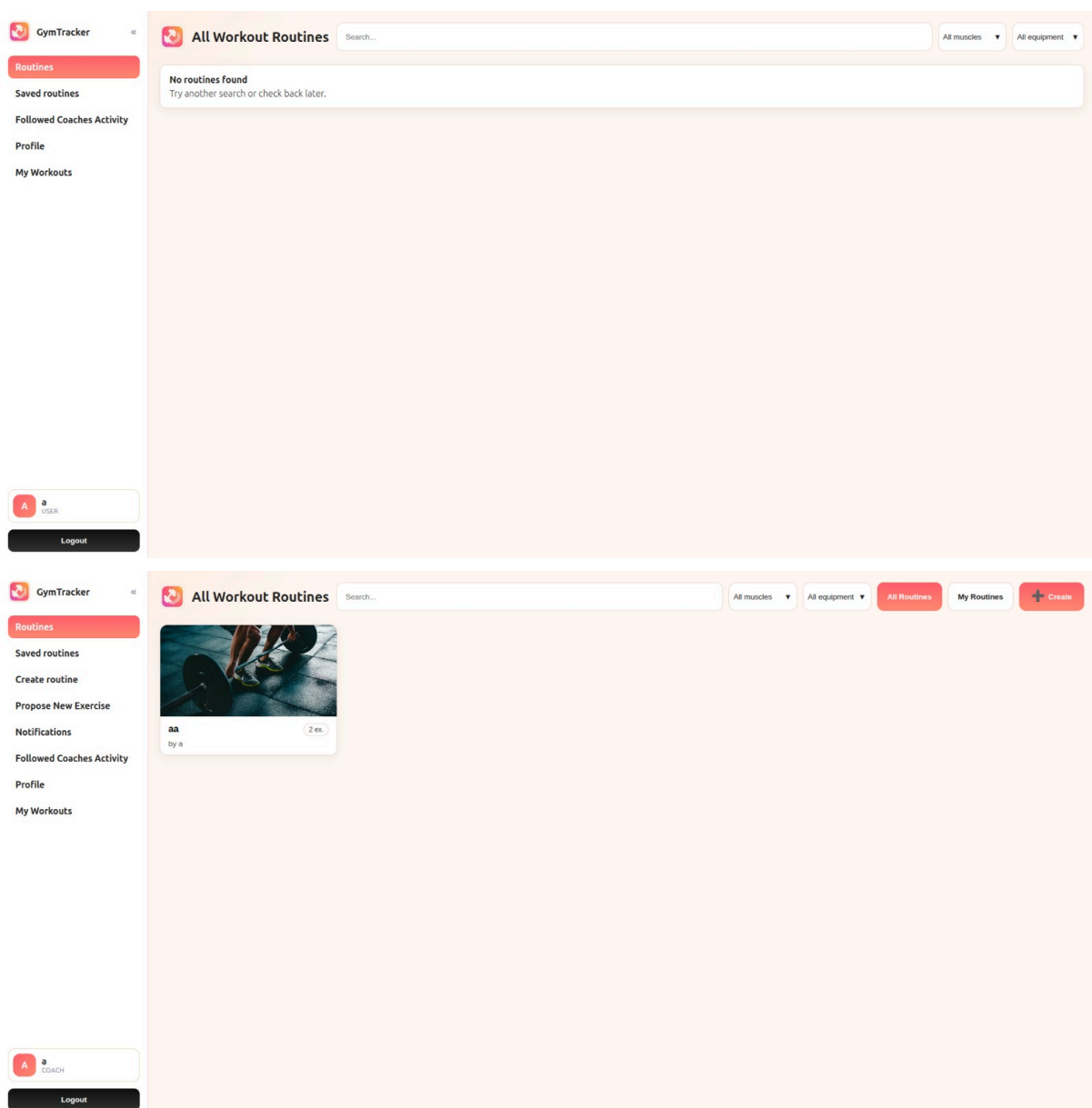
En conclusión, las expectativas de calidad planteadas al inicio del proyecto se han cumplido en gran medida, y el resultado final refleja un progreso significativo en la aplicación de buenas prácticas de desarrollo y uso de herramientas de soporte al ciclo de vida del software, al igual que un uso correcto y progresivo de las herramientas de desarrollo proporcionadas.

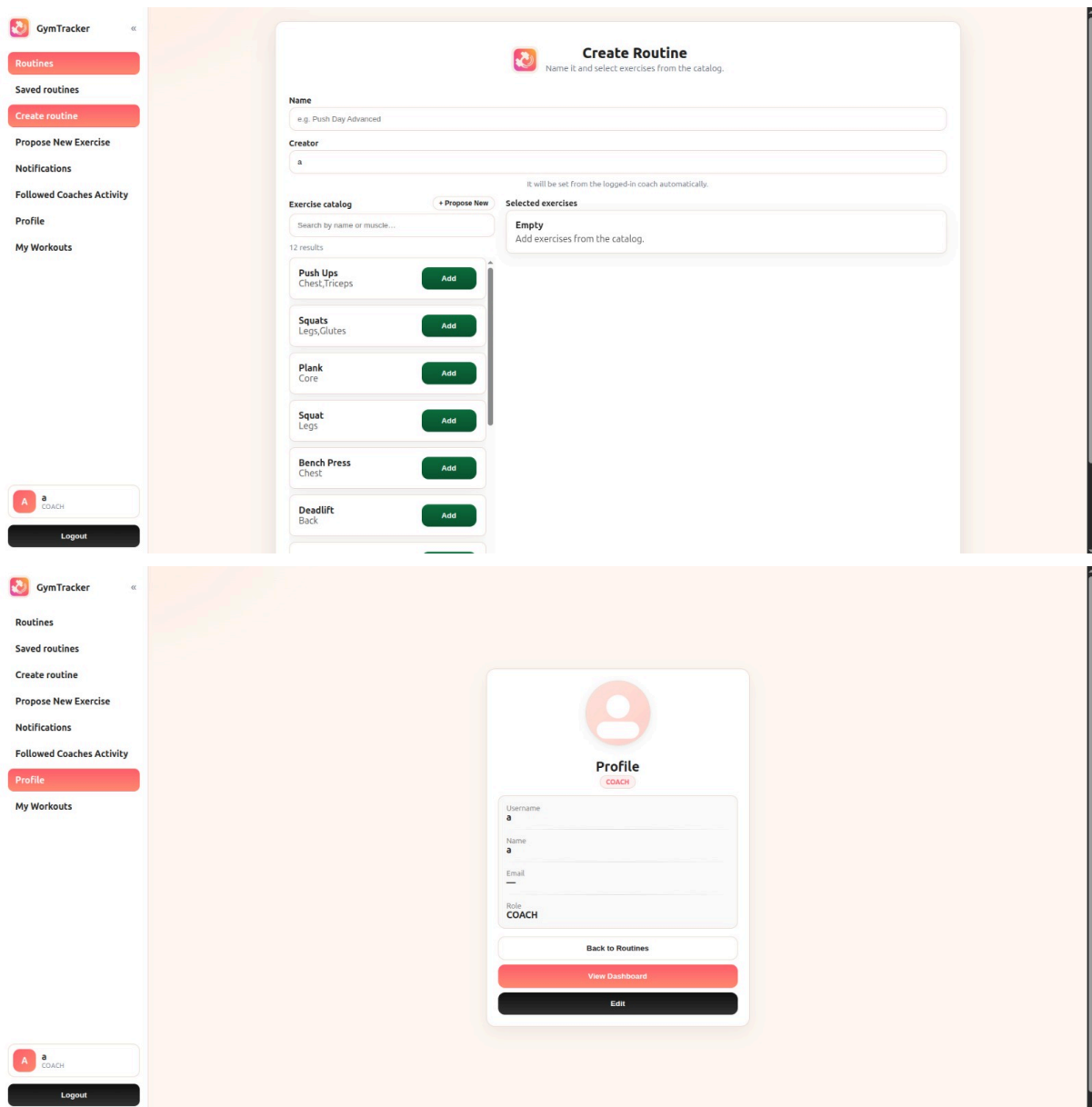
7. Evolución del proyecto

En este apartado se va a ver como ha sido el progreso en la aplicación, viendo cómo se pasó de algo muy sencillo a un proyecto más complejo.

En la primera iteración tuvimos el problema de borrar las ramas merengadas, entonces perdimos la trazabilidad de la primera iteración.

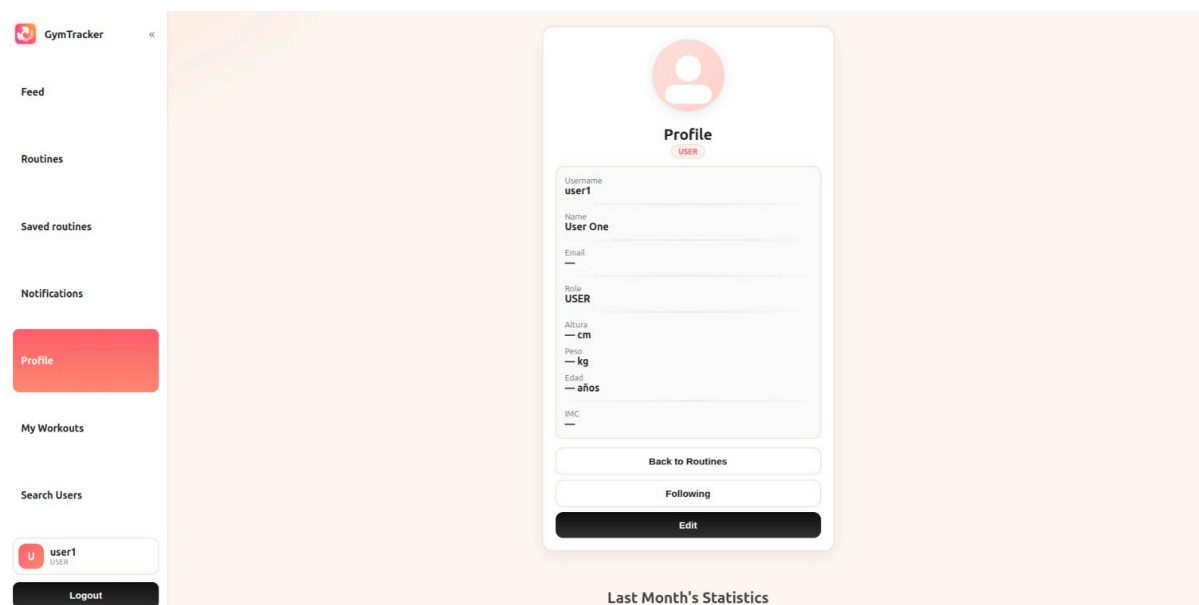
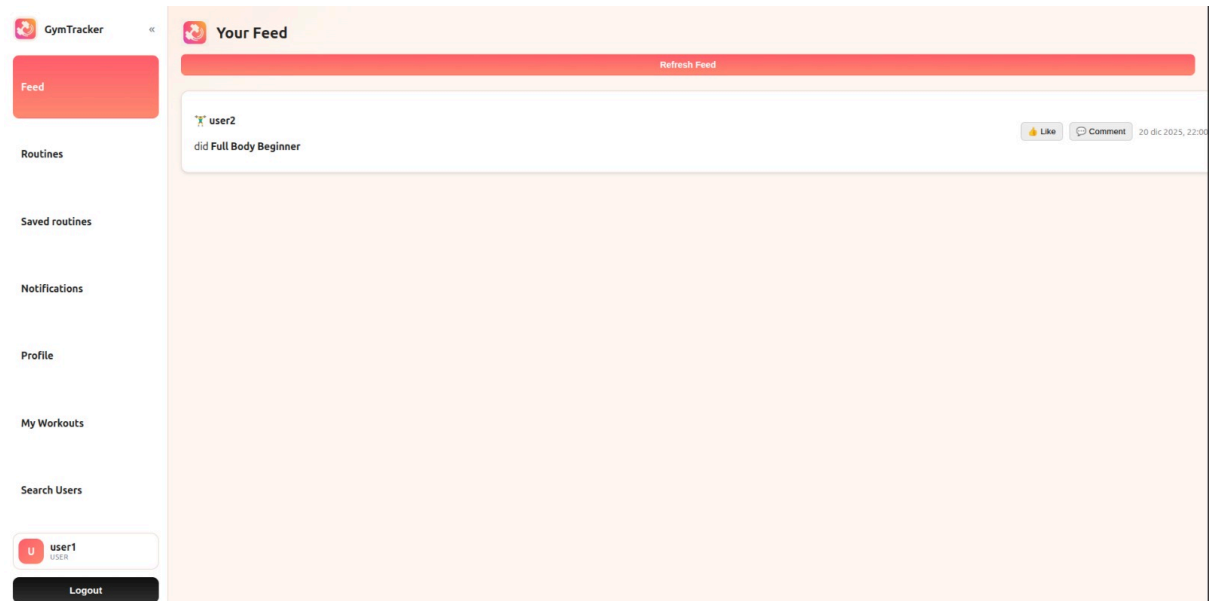
7.1. Segunda iteración (SCRUM)

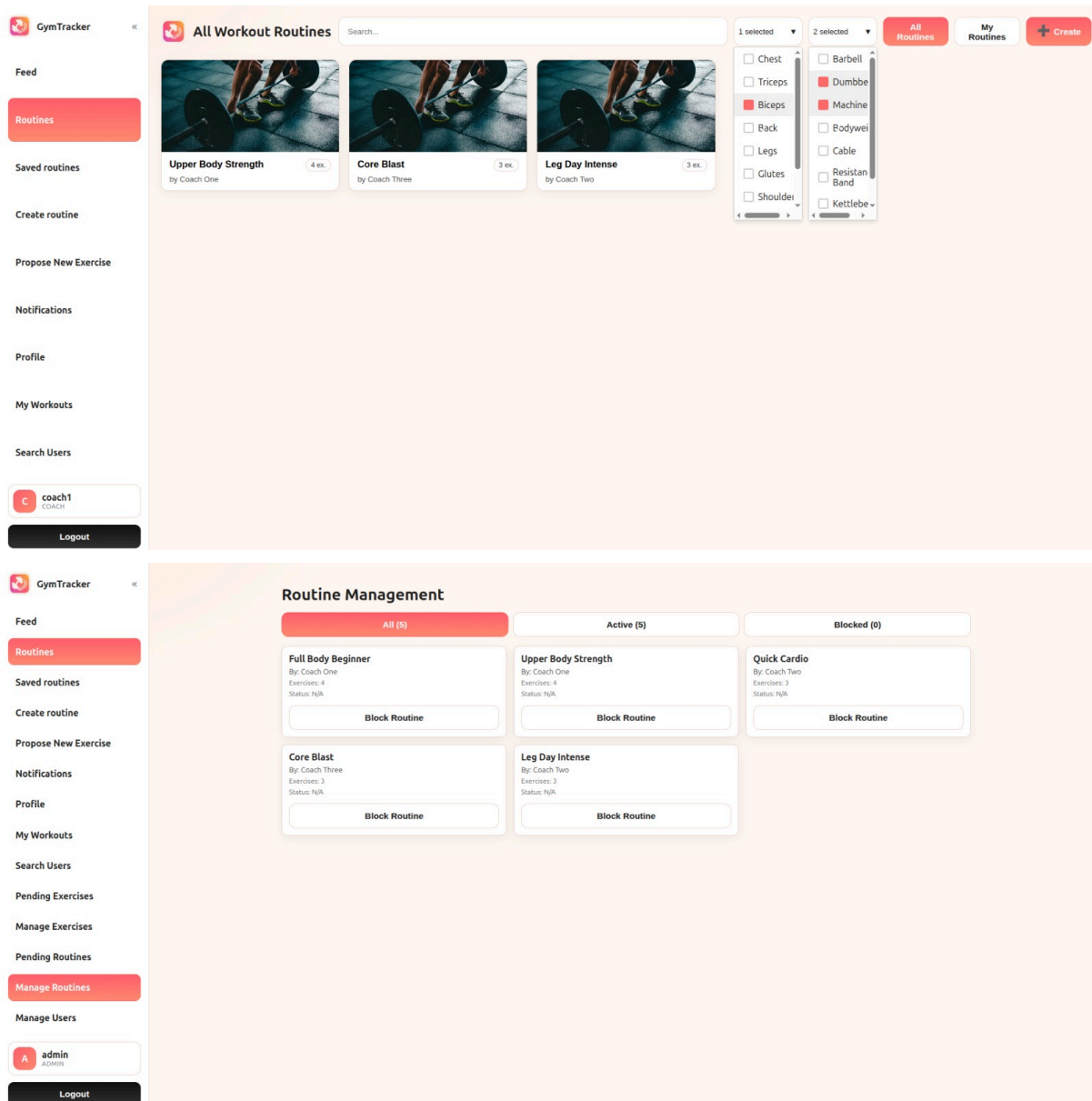




Se puede observar que era una versión temprana, en la que no había datos de ejemplo, parecía más vacío y carecía de más características que se fueron añadiendo posteriormente.

7.2. Tercera iteración

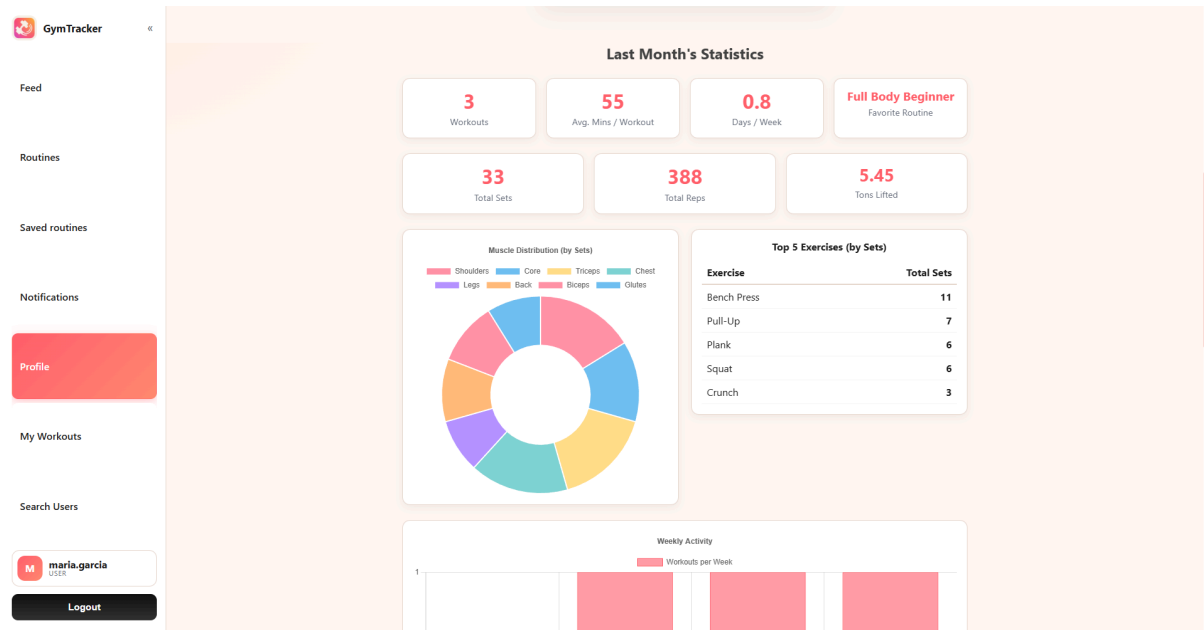


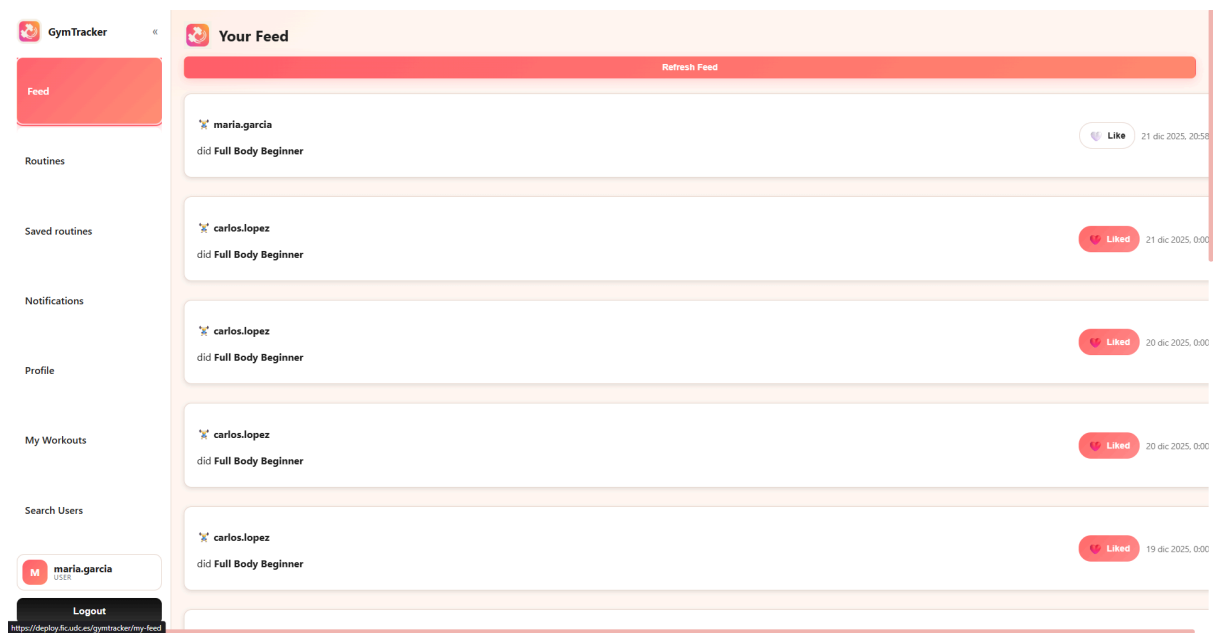
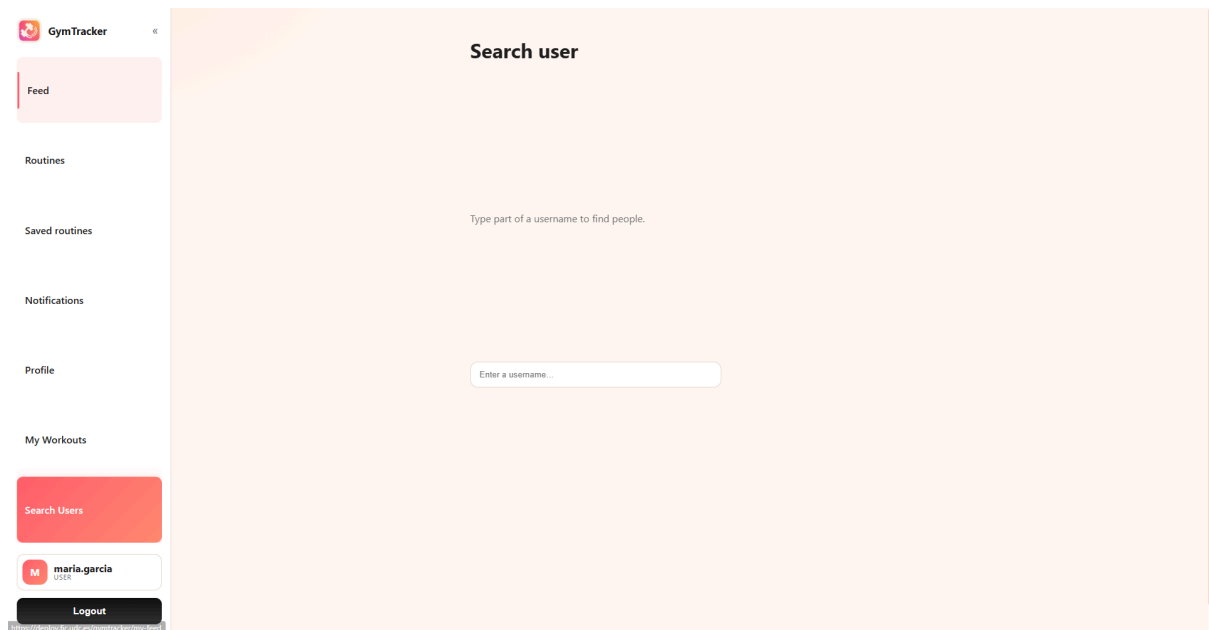


En esta iteración es donde se hicieron los cambios más significativos a nivel de funcionalidades, la introducción de la feed, que fue uno de los retos más grandes como se comentó anteriormente en el informe.

Aplicar los filtros, el rol de admin, que fue algo que tardamos en añadir de manera definitiva, y que tiene el control total de la app, el perfil de usuario cambiado y mejorado, los like y los comentarios fueron algunas de las cosas añadidas y son de las cosas más significativas en relación a los cambios que tuvo el programa y se realizaron todas en este iteración.

7.3. Cuarta iteración





Anteriormente se dijo que la tercera iteración es la que más significó a nivel desarrollo interno, y esta es la que más significó a nivel visual, animaciones, colores y un perfeccionamiento visual que hace que suba mucho el atractivo para los clientes.

Se puede observar una feed más compleja. más datos introducidos, haciendo que se refine la aplicación. La opción de buscar el usuario, que hace una función clave en el aspecto social de la aplicación, unas estadísticas, gráficos y leaderboards más visuales y complejos, que hace que se complete la aplicación y concluya el proceso de desarrollo de GymTracker.